ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

System resiliency

Michele Colajanni

Dipartimento di Informatica – Scienza e Ingegneria

michele.colajanni@unibo.it

Resilient material?

Steel

Glass

Sponge

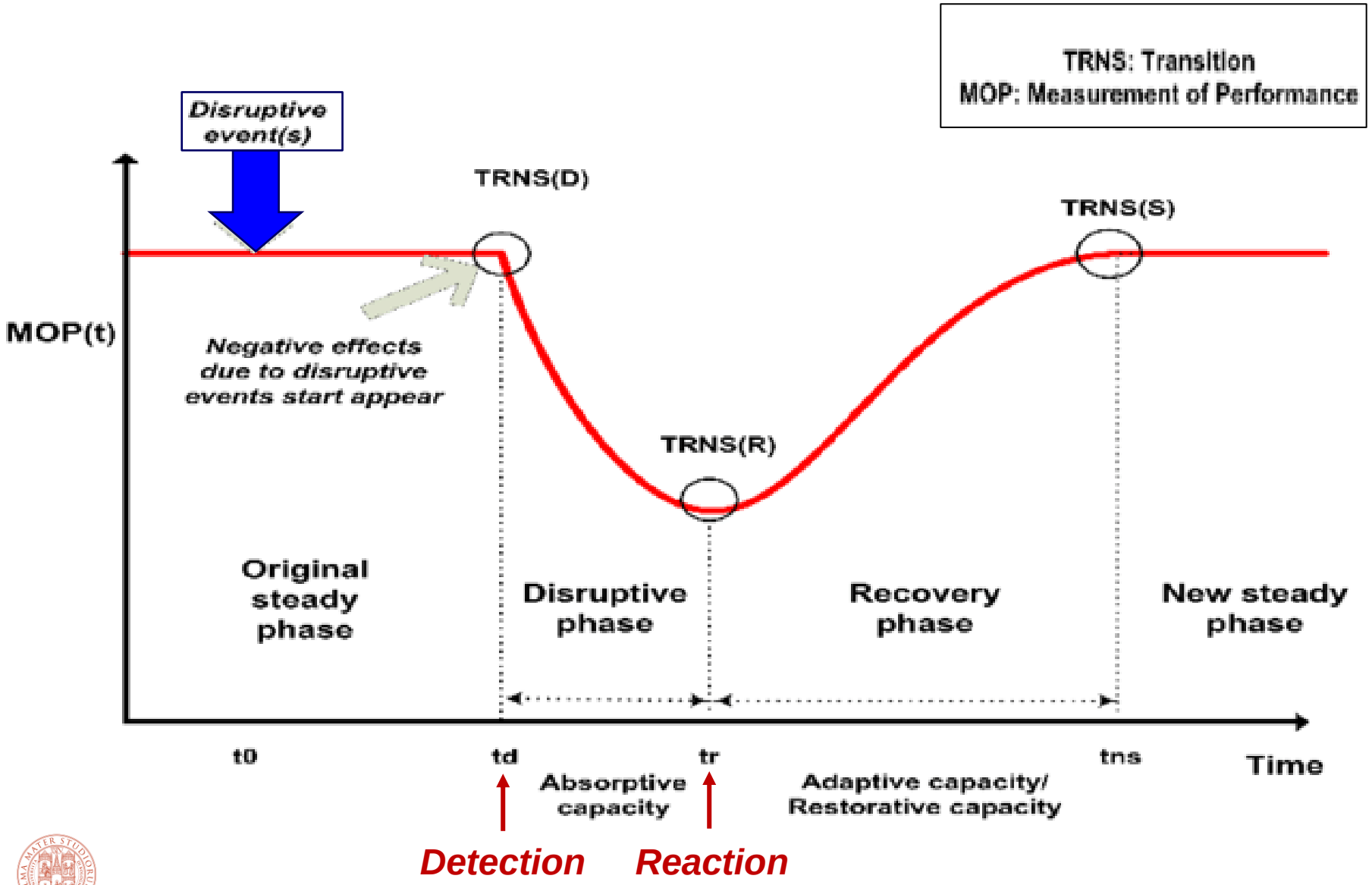
Resiliency

Many disciplines adopt ***resiliency principles***, including:

- Engineering 
- Psychology
- Economy
- Health system
- Ecosystem
- Cities (e.g., Rockefeller Foundation's City Resilience Framework)
- Society

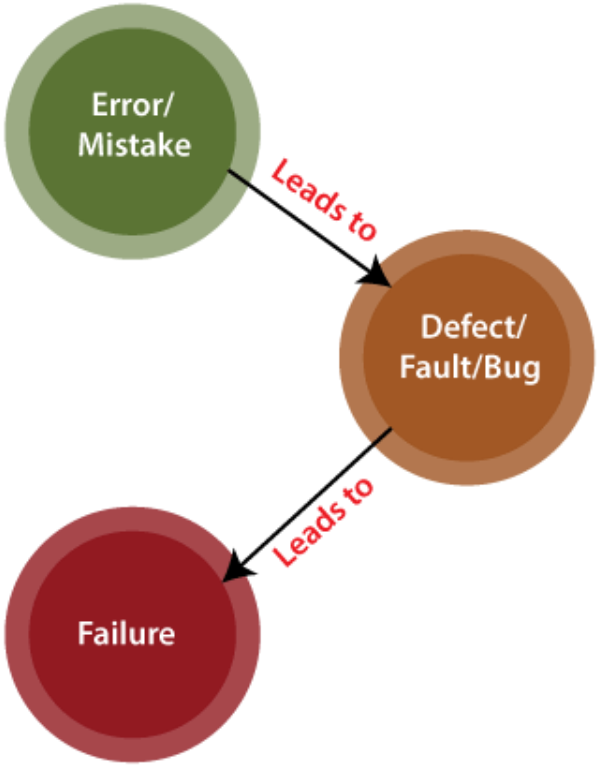
All of them have to do with **EVENTS** against normal operation and consequent ways for **managing negative effects**

Resilient system



Definitions (*for specialists*)

- **Error**: you know why the fault occurred
- **Defect**: you know something is not working as per the requirement. A software defect is a **bug**
- **Fault**: you know the cause, but you do not know why the fault occurred. A software fault is a state that causes the software to fail to accomplish its essential function
- **Failure**: you know something is wrong but you do not know the cause. A software's failure causes a fatal issue in an application or in its module, which makes the system unresponsive or broken



Important terms

- **Vulnerability** is a weakness or a state of susceptibility which opens the infrastructure to a possible outage due to attack or circumstance
- A ***system fault*** is the cause of a triggered vulnerability, or error state
- A ***threat*** is the potential for a vulnerability to be exploited or triggered into a disruptive event

Vulnerabilities or faults, can be exploited intentionally or triggered unintentionally

Reliability *vs*. Resiliency

Reliability is the quality of being trustworthy or of performing consistently well

It is typically associated with a well designed and implemented infrastructure and refers to the probability that the system will work as expected

- A system may be available for a certain period of time, but it may not work as expected for providing requested services. In that case, the reliability will be low.

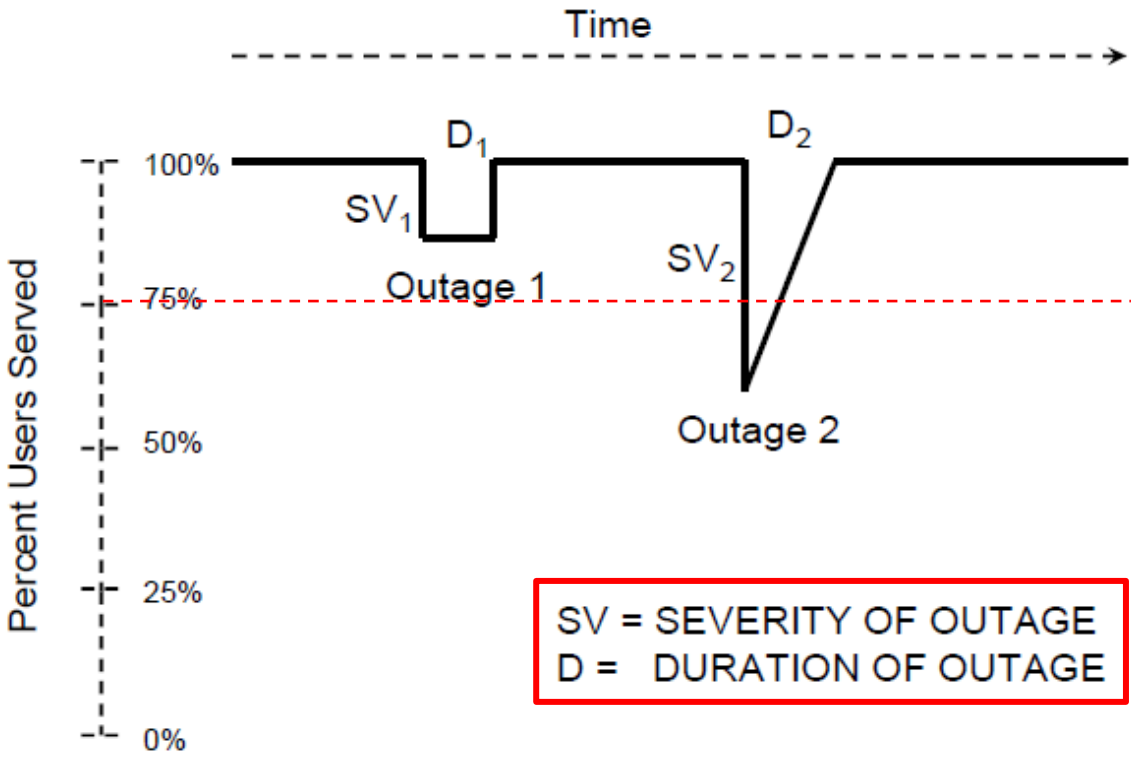
Resilience is the capacity to recover quickly from difficulties

- The focus is not on prevention, but on incident management and severity
- Some texts prefer the term **maintainability**, but its focus in on the MTTR of the reliability context

Measure for resilience

- One way to measure resiliency is to set a **severity threshold** and observe the fraction of time the infrastructure is in a resilient state
- **An incident is not a transient event**. At any instant in a system there are a small number of users without an acceptable service. Which is the **acceptable percentage** (*threshold*) and **duration** that can distinguish a *resilient state* from a *not resilient* state?
- Hence, we can define **resiliency as the fraction of time the infrastructure is in a resilient state**:
  - **MTTRD** is mean time to resiliency deficit (RD)
  - **MTTRD** is mean time to restore the resiliency deficit D

Example: possible outage profiles



SV = SEVERITY OF OUTAGE
D = DURATION OF OUTAGE

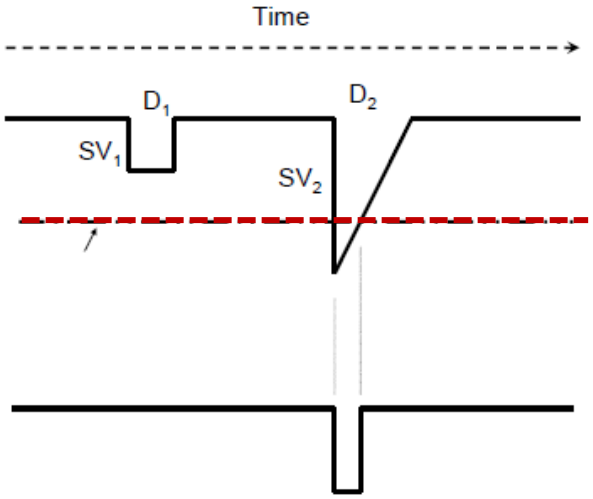
Outage 1: Failure and complete recovery. E.g. Switch failure

Outage 2: Failure and graceful Recovery. E.g. Fiber cut with rerouting

Resiliency threshold

- **Resiliency** describes the ability of a system to self-heal, recover, and continue operating after encountering failure, outage, cybersecurity incidents, etc.
- **High resiliency** does not mean a **high availability**. It means that the infrastructure and the organization are well equipped to overcome disruptions
- Resiliency is in some sense related to the **Mean Time to Repair** (**MTTR**), which captures how long it takes to get the infrastructure up and running after a failure: lower the MTTR, better the resiliency

$$S = \frac{MTTRD}{MTTRD + MTTRD}$$



System resiliency

The **resilience** of a system is an evaluation of how well that system can maintain the **continuity of its services even** in the presence of disruptive threats, such as

- cyberattacks by malicious and competent actors
- serious equipment failure
- external factors such as natural or systemic events

We have to consider cyber threats, but not only …

INTENTIONAL ACTION

- External attacks
- Supply chain attacks
- Internal attacks

NEGLIGENT ACTION

- Human errors

Many threat types

INTENTIONAL ACTION

External attacks

Supply chain attacks

Internal attacks

NEGLIGENT ACTION

Human errors

Cybersecurity

INTERNAL ACCIDENTAL EVENT

Failures

Incidents

Black out

Network and/or system outage or malfunction

Water leak

Short circuit and fire

SUPPLIER ACCIDENTAL EVENT

Failures

Incidents

EXTERNAL ACCIDENTAL EVENT

Natural disasters

Flooding, fire, extreme weather, earthquakes, …

EXTERNAL (HUMAN) EVENT

Changes in Law

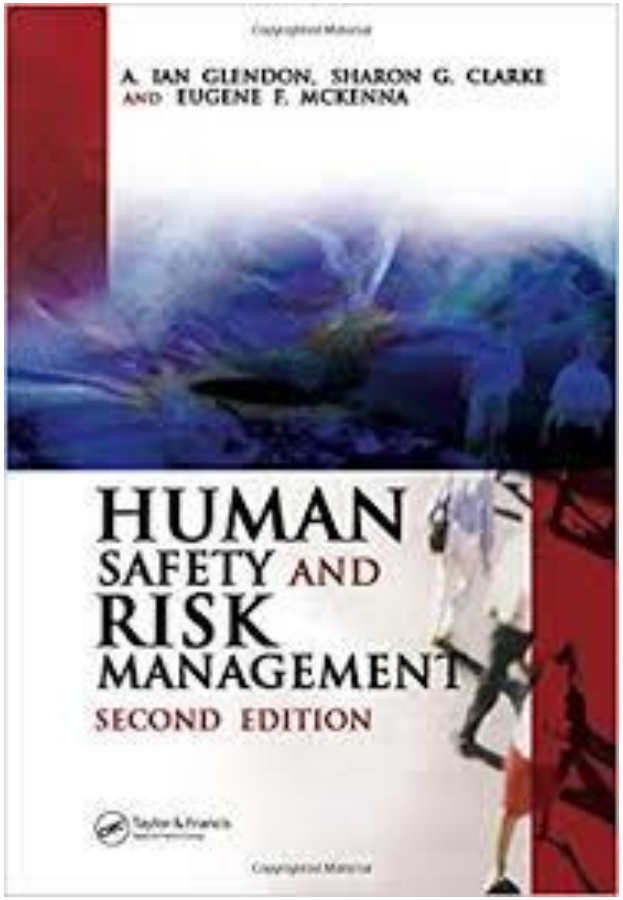
Changes in market rules

Technological innovations

The human race is more resilient than reliable

Risk management is likely the <u>best</u> innate human skill

- Our ancestors were able to manage <u>personal risks</u> (***safety***)
- Capacity of actors, institutions, and populations to:
  - Prepare for effectively responding to crises
  - Maintain core functions when a crisis hits
  - Recovery
  - Learned lessons during the crisis including possible reorganization if conditions require it

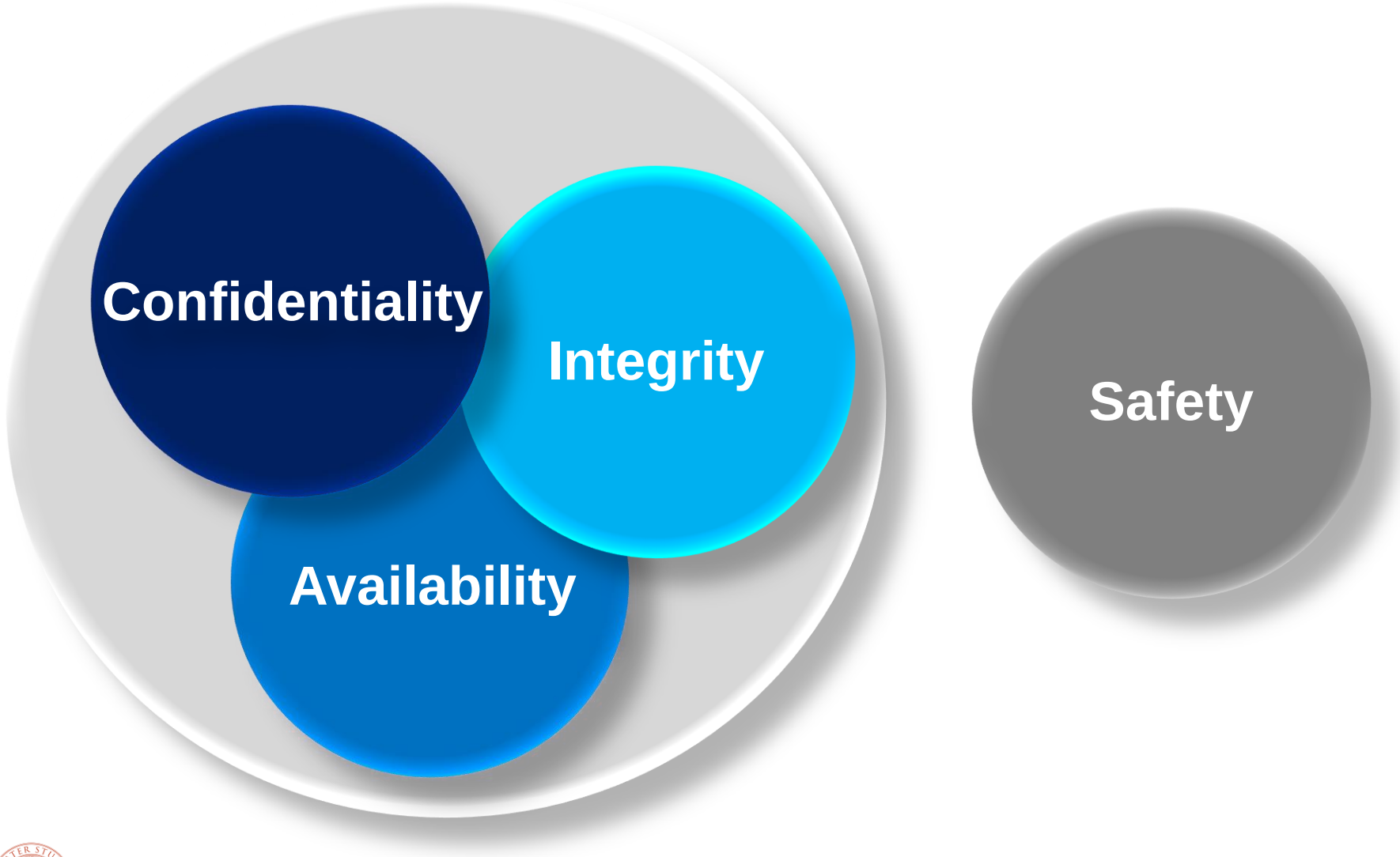


Focus of traditional resilience disciplines

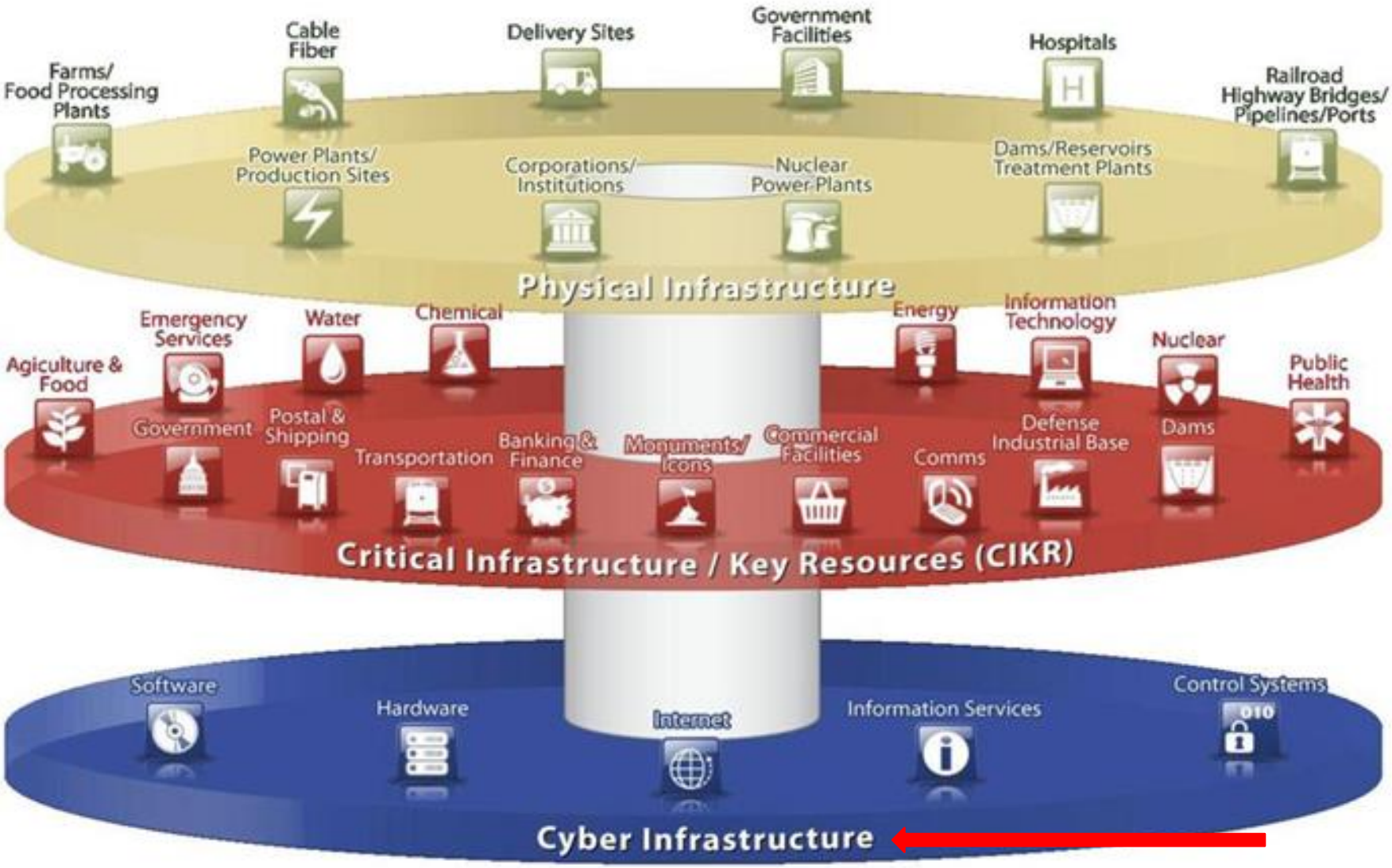
The human (and social) condition of being ***safe*** from suffering hurt, injury, or loss

Focus of IT disciplines

Focus of cyber-physical systems (CPS)

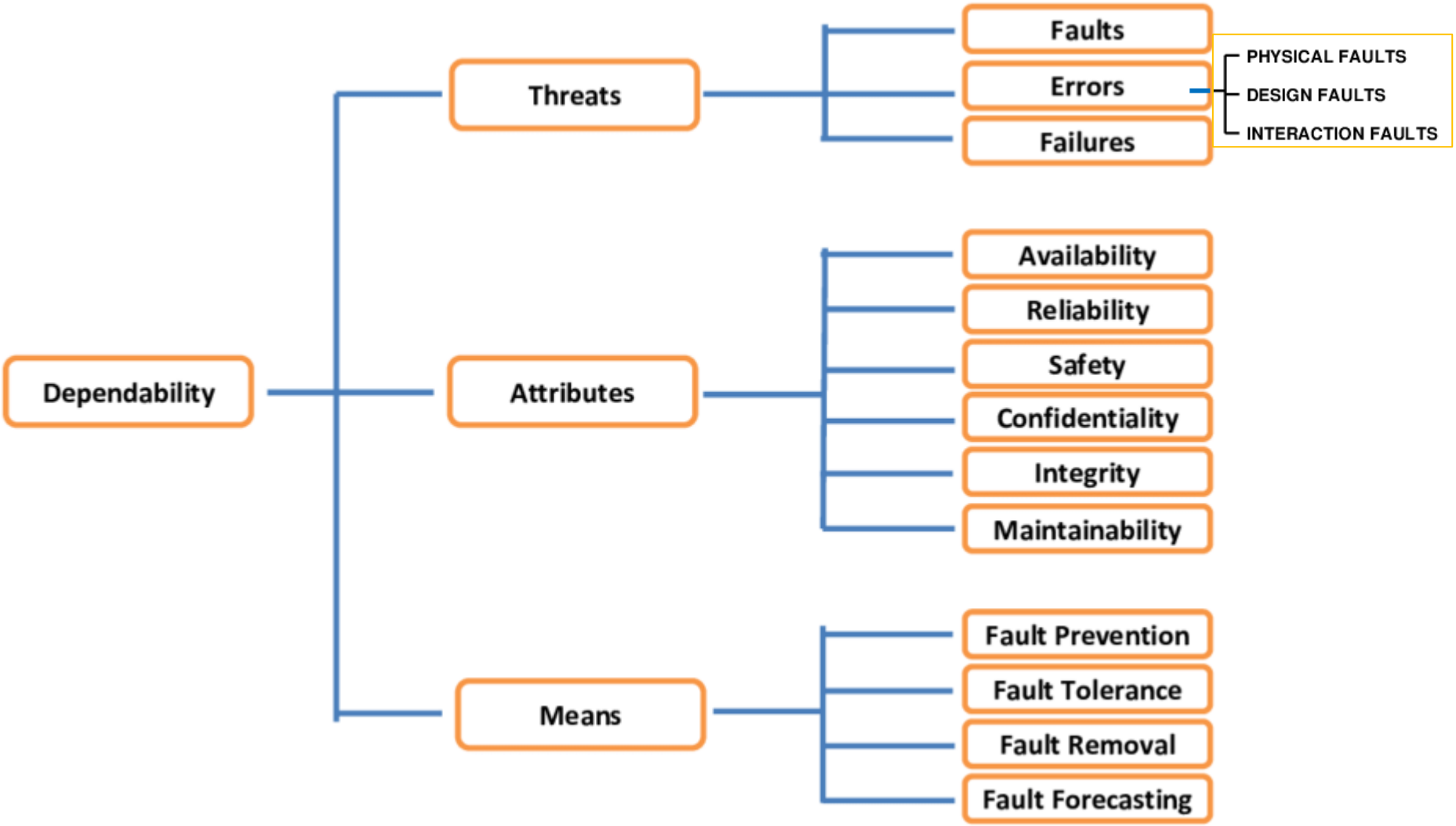


View of modern system based on cyber infrastructures



Design for resilience

The taxonomy *Dependability framework* for engineers



A. Avizienis, et al, "Basic Concepts & Taxonomy of Dependable & Secure Computing", *IEEE Transactions on Dependable & Secure Computing*, 2004

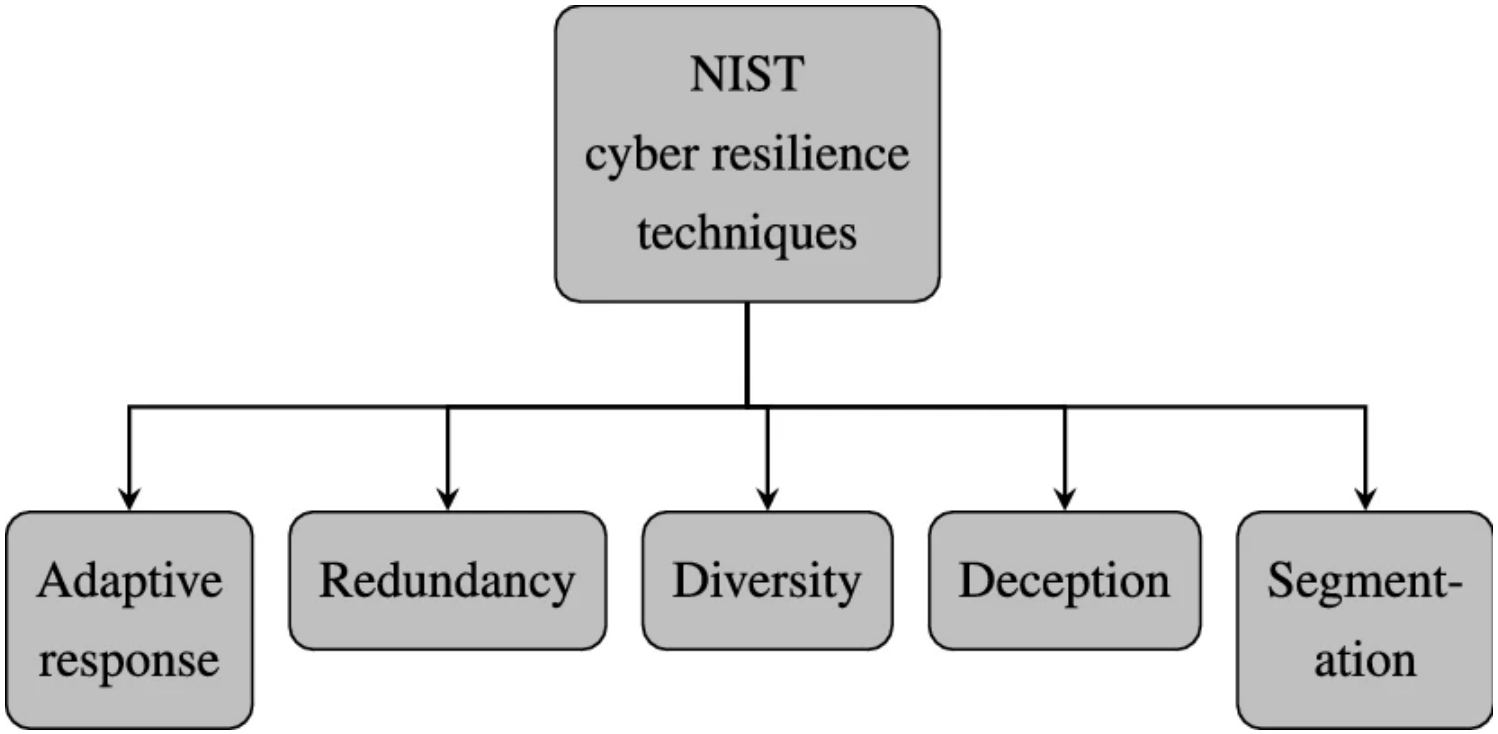
System resiliency

Resiliency is not a standalone function; it spans:

- Business continuity
- Incident response
- Disaster recovery techniques
- Incident management

to reduce the magnitude and duration of disruptive events

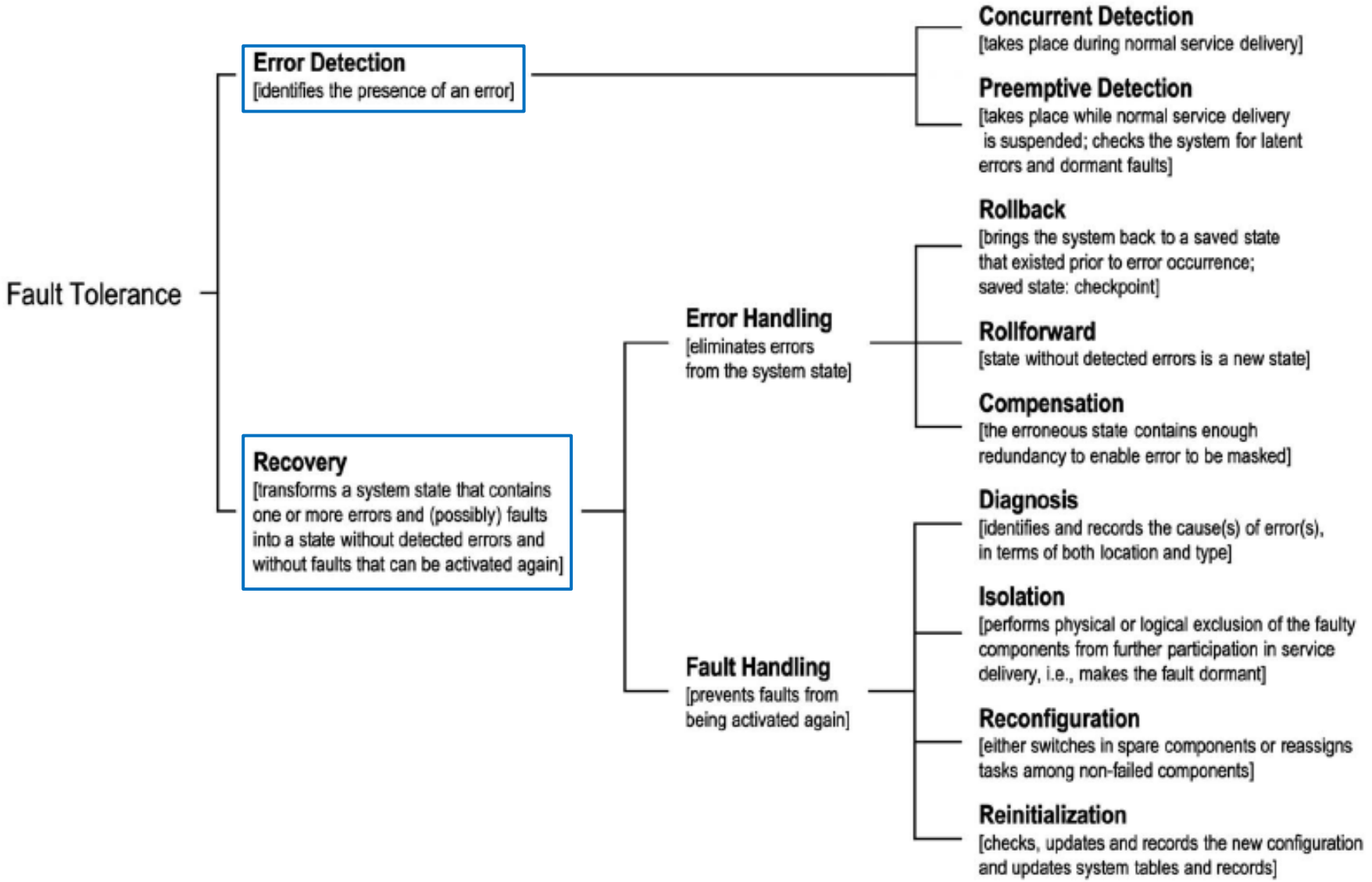
Suggested attributes by NIST



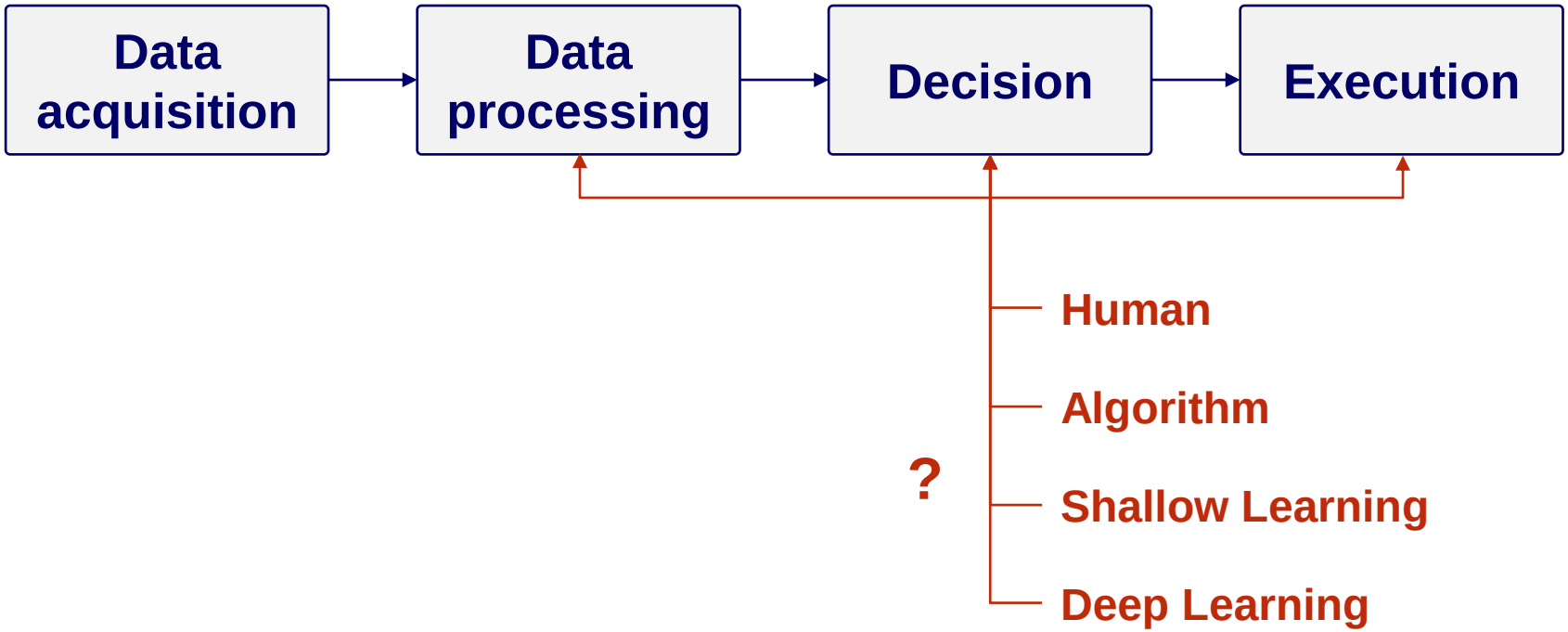
Three main qualities for system resilience

- **Robustness**: being unaffected by disturbance
- **Adaptability**: being able to diagnose a disturbance and adapt itself to be unaffected by the disturbance
- **Maintainability**: being organized to enable rapid and often improved abilities to be unaffected by a disturbance. Maintainability includes
  - **Modifiability**: easy and speed of modification (also in terms of rapid adaptability to new opportunities and threats)
  - **Repairability**: easy and speed of fixing defects and vulnerabilities

How to achieve *resilience* in terms of tolerance to faults



The key points



***Resiliency must be guaranteed in each phase and for each possible actor*: human, algorithm, AI !**

4Rs for Resilience

- **Recognition**: The system or its operators should recognize early indications of system failure
- **Resistance**: If the symptoms of a technical problem or of a cyberattack are detected early, then resistance strategies may be used to reduce the probability that the system will fail
- **Recovery**: If a failure occurs, the recovery activity ensures that (some) critical system services are restored quickly so that system users are not badly affected by failure
- **Reinstatement**: All the system services are restored, and normal system operations can continue

My “R”s for a Resilient system

PREVENTION

- **Reliability**: Guaranteeing that the components are inherently designed to work under a scope of conditions (component’s view)
- **Resistance**: Prevent damage or interruption by giving the quality or assurance to struggle the risk or its essential effect (component’s or system’s view)
- **Redundancy**: Provide backup configurations or spare capacity to enable or switch operations to alternate methods (system’s view)

DETECTION

- **Recognition**: The system or its operators should recognize early indications of anomaly and system failure
- **Resistance**: If the symptoms of a technical problem or of a cyberattack are detected early, then resistance strategies may be used to reduce the probability that the system will fail

RESPONSE

- **Recovery**: If a failure occurs, the recovery activity ensures that (some) critical system services are restored quickly so that users are not badly affected by failure
- **Reinstatement**: All of the system services are restored from problematic events, and normal system operations can continue

Resilience engineering

Resilience engineering assumes that it is impossible to avoid system failures and cyberattacks

(especially for a large distributed, networked system)

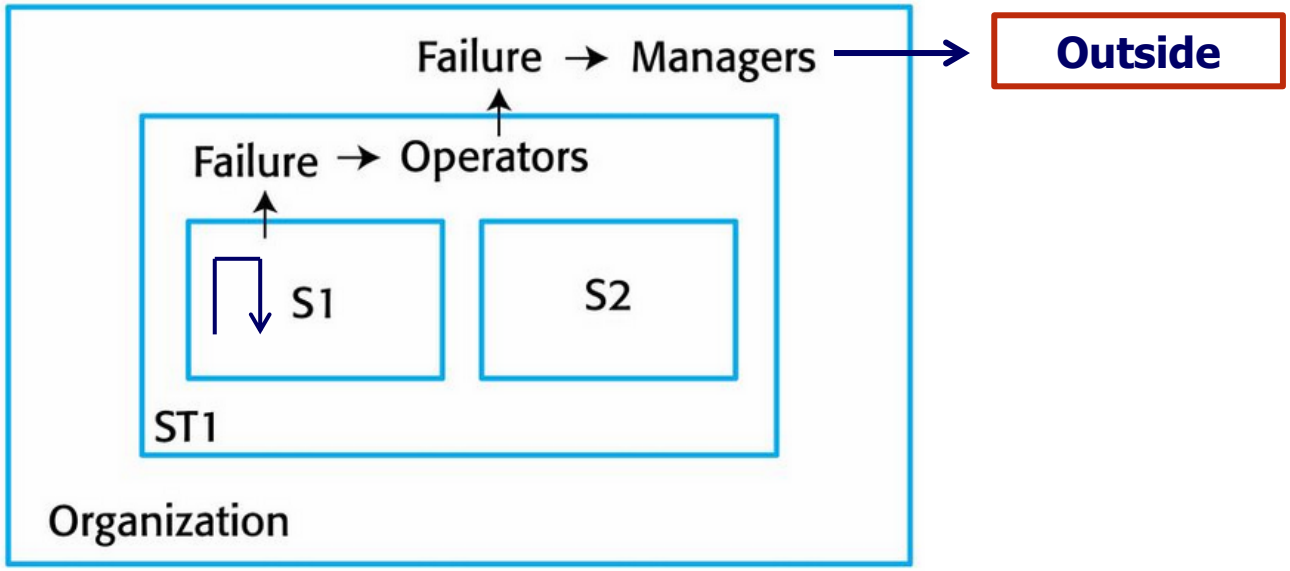
- The traditional focus was on good **reliability engineering** practices to minimize the number of technical faults in a system
- Today, **reliability engineering** places emphasis also on limiting the human factors, such as operator errors and cyberattacks

Resilience engineering is also concerned with:

1) **limiting the effects of these events**
2) **recovering from them**

System resiliency: technology + management (Example: *Failure escalation*)

1. Some failures can be automatically contained by the system (S1)
2. Other failures in system S1 may be trapped in the broader socio-technical system ST1 through Operator actions. Organizational damage is therefore limited
3. If the failure in S1 leads to a failure in ST1 that cannot be dealt by Operators, then it scales up to Managers in the broader Organization
4. Some (few) cases scale to outside requiring external help



Everything by design

- Privacy by design
- Security by design
- Scalability by design
- Resilience by design
  - Detection by design
  - Resistance by design

Both **scalability** (*elasticity in performance terms*) and **resilience** has to do with a common attribute: **elasticity** of the system

➔ So, the real target in this continuous changing world and business market is to create ***adaptable* systems that are** managed by ***adaptable* organizations**

The target: *Adaptability*

The system ability to change to fit *altered* circumstances

Even personal
(the top skill for the 21st century – *consider them in interviews*):

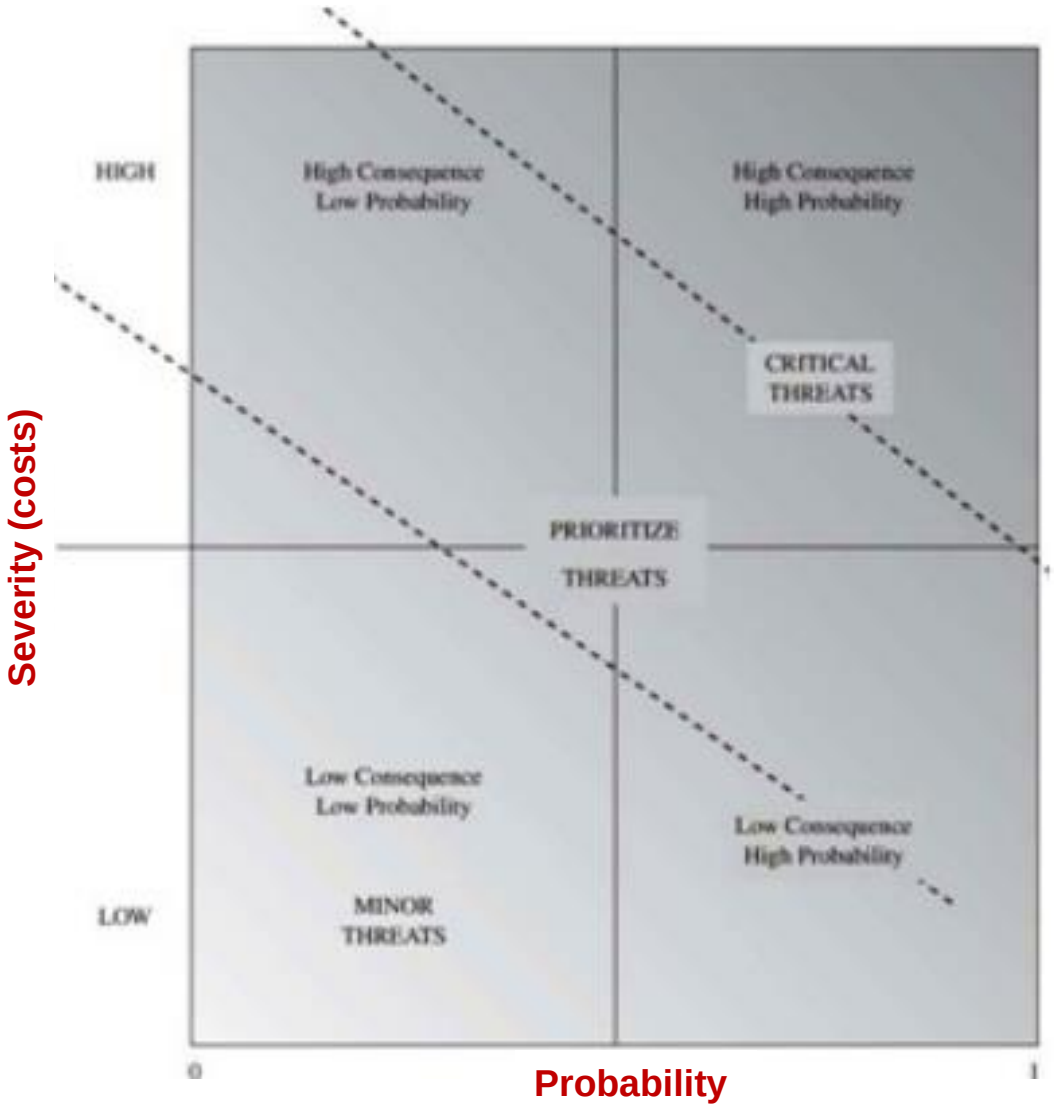
1) The ability to **adjust** your emotions, thoughts and behaviors to changing situations and conditions
2) Being **open** to changes, new ideas, challenges and methods

Difficult! *And for this reason, strongly desired*

Resilience engineering – *Key points*

1. Resilience is a judgment of how well a system can maintain the continuity of its (critical) services in the presence of disruptive events
2. **Business resilience requirements** should be the starting point for designing resilient systems
3. System resilience should be based on the **R's model**
4. **Resilience planning** should assume the possibility of faults and attacks, and that some of them will be successful
5. Systems should be designed with **defensive layers** of different types. These layers trap human and technical failures and help resist attacks
6. To allow system operators and managers to cope with problems, processes should be flexible and adaptable. **Process automation can both help and make it more difficult for people to cope with problems**
7. **You cannot do it all**. An important part of system resiliency design is identifying the **most critical services**. Systems should be designed so that these critical services are protected and, in the event of failure or attacks, are recovered as quickly as possible

Risk management



Services are based on a network of components

- **How do we calculate the probability of a cyber attack or of a failure?**
- **How do we recognize the most critical vulnerabilities?**

- The theory and practice of **Risk management** exist because you cannot afford to face and stop every possible fault or attack
- One important measure for priority is the **Expected loss** (**Probability** of a negative event x **Cost** of the damage)

Assessing Risk is Difficult

Severity

- Economic impact
- Geographic impact
- Safety impact

Probability

- Vulnerabilities
- Means and Capabilities
- Motivations

Quantitative probabilities have little utility for rare events (including multiple concurrent outages)

Severity

The measure of severity can be expressed in several ways, e.g.,

- Percentage or fraction of users potentially or actually affected
- Number of users potentially or actually affected
- Percentage or fraction of offered or actual demand served
- Offered or actual demand served

The distinction between ***potentially*** and ***actually*** affected is important:

- If a 100,000 switch were to fail and be out from 3:30 to 4:00 am, there are 100,000 users potentially affected
- However, if only 5% of the lines are in use at that time of the morning, 5,000 users are actually affected

Minimizing severity of outages

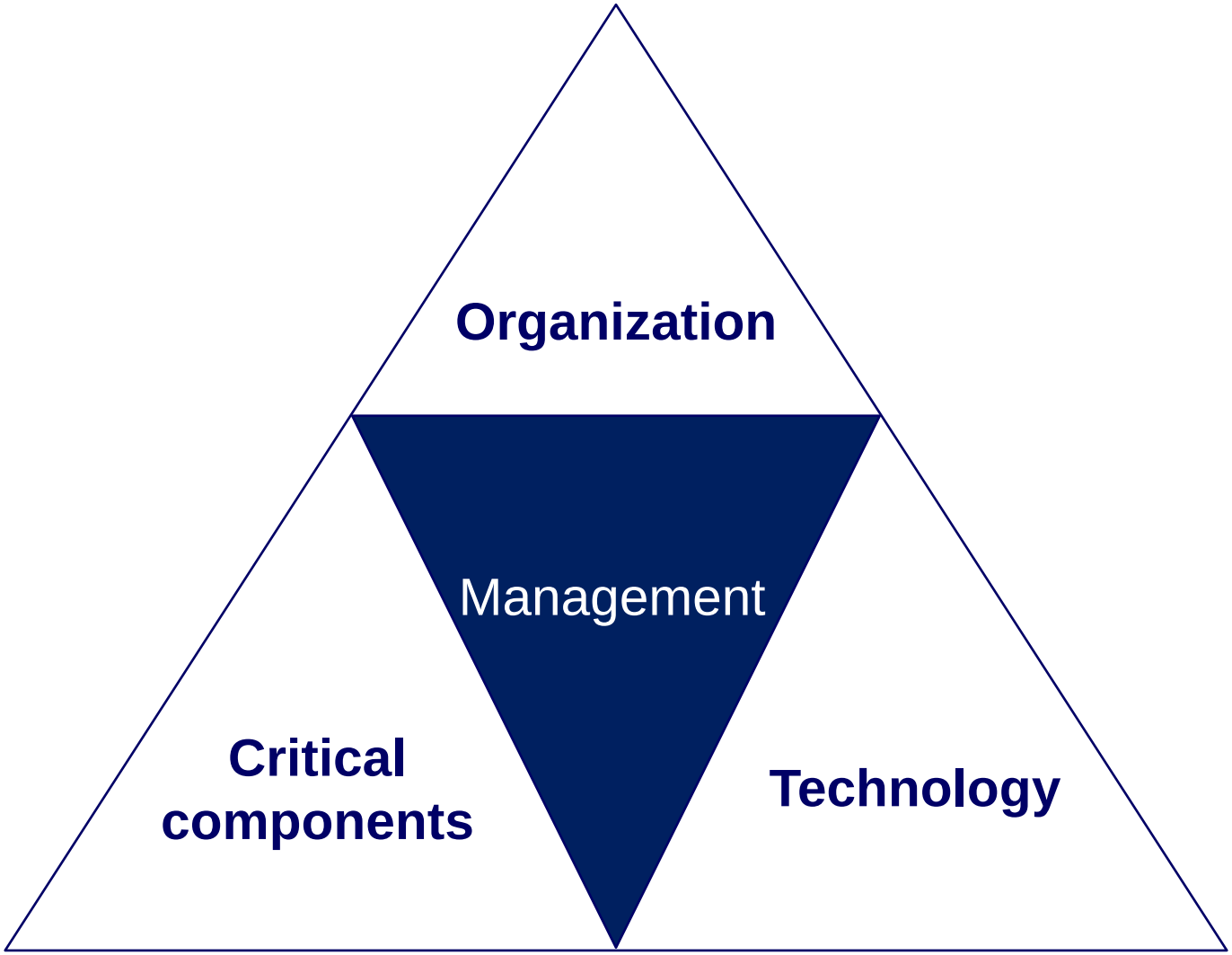
It is not always possible to completely avoid failures that lead to outages. Proactive steps can be taken to minimize their size and duration.

- Avoiding single points of failure that can affect large numbers of users
- Having recovery assets optimally deployed to minimize the duration of outages

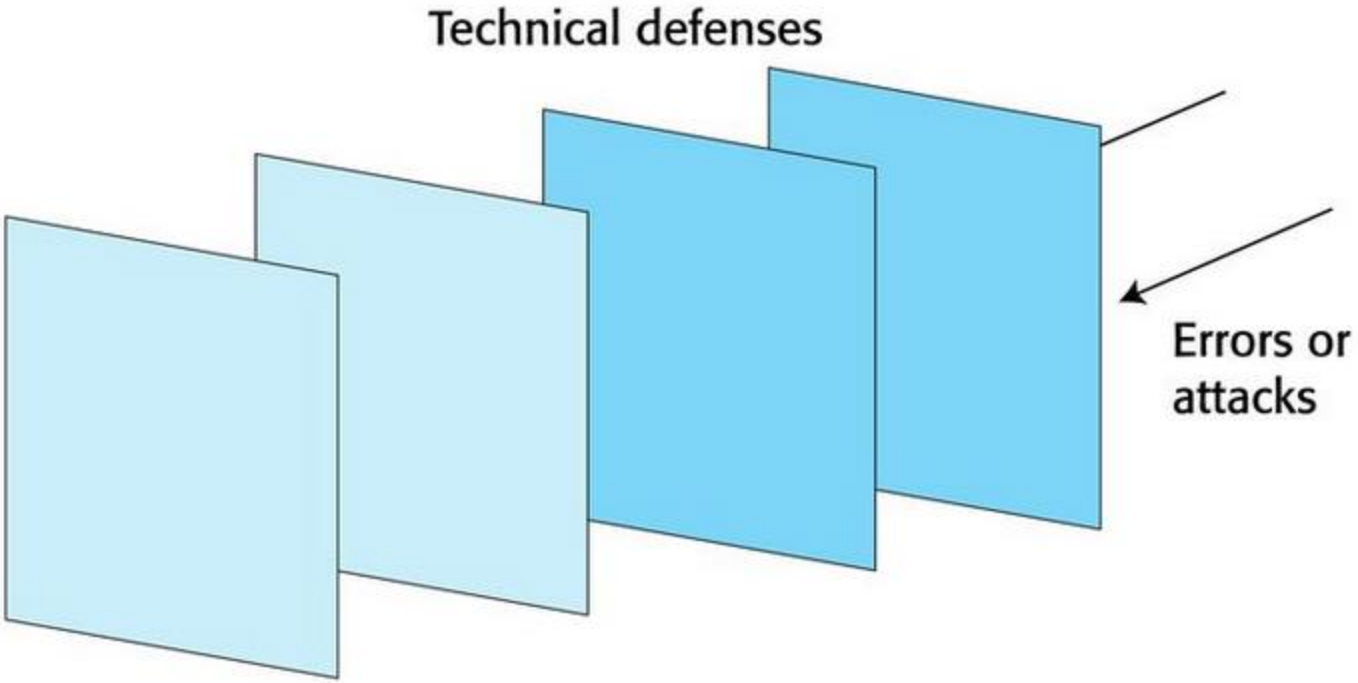
This can be accomplished by:

- Ensuring there is not too much over-concentration of assets in single buildings or complexes
- Properly deploying and operating fault tolerant ICT architectures
  - Equipment/power fault tolerance
  - Physically and logical diverse transmission systems/paths
- Ensuring there is adequately trained staff and dispersal of maintenance capabilities and assets

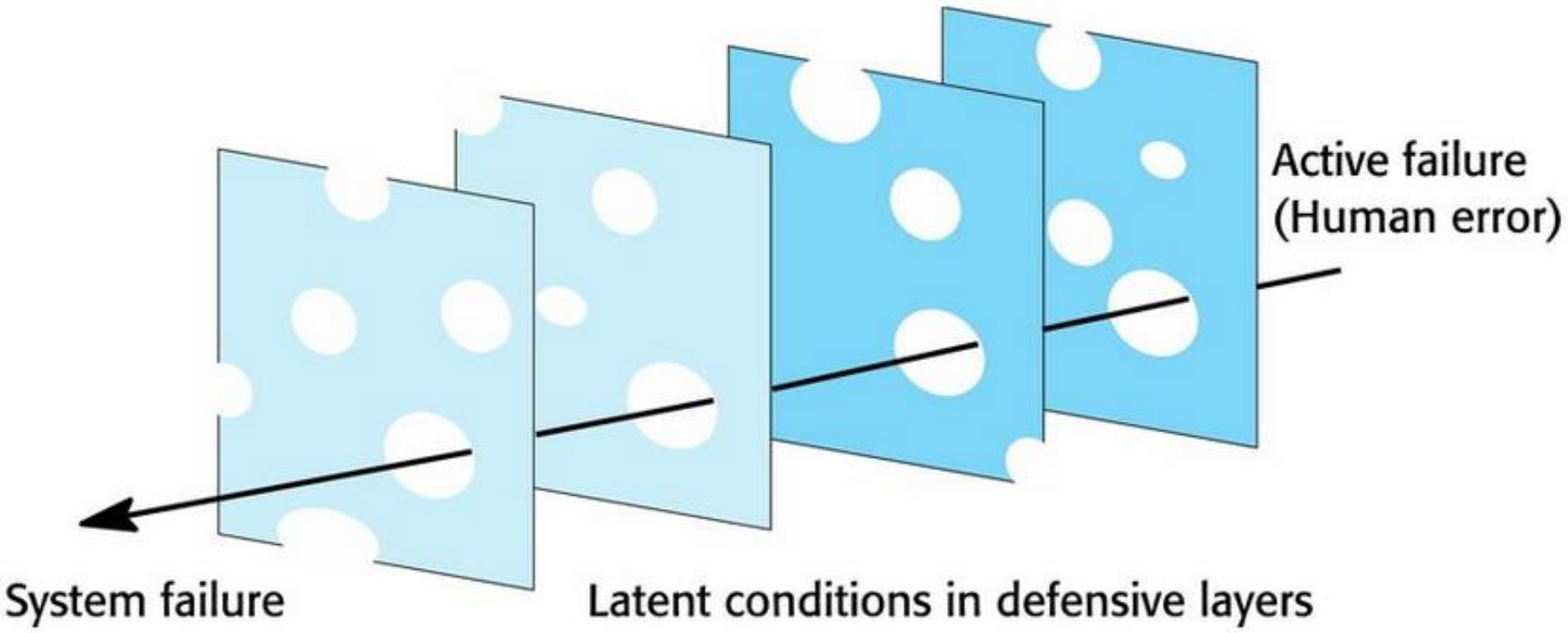
Scalability and Resiliency depend on the strengths of three main cornerstones



Defensive layers (*ideal*)

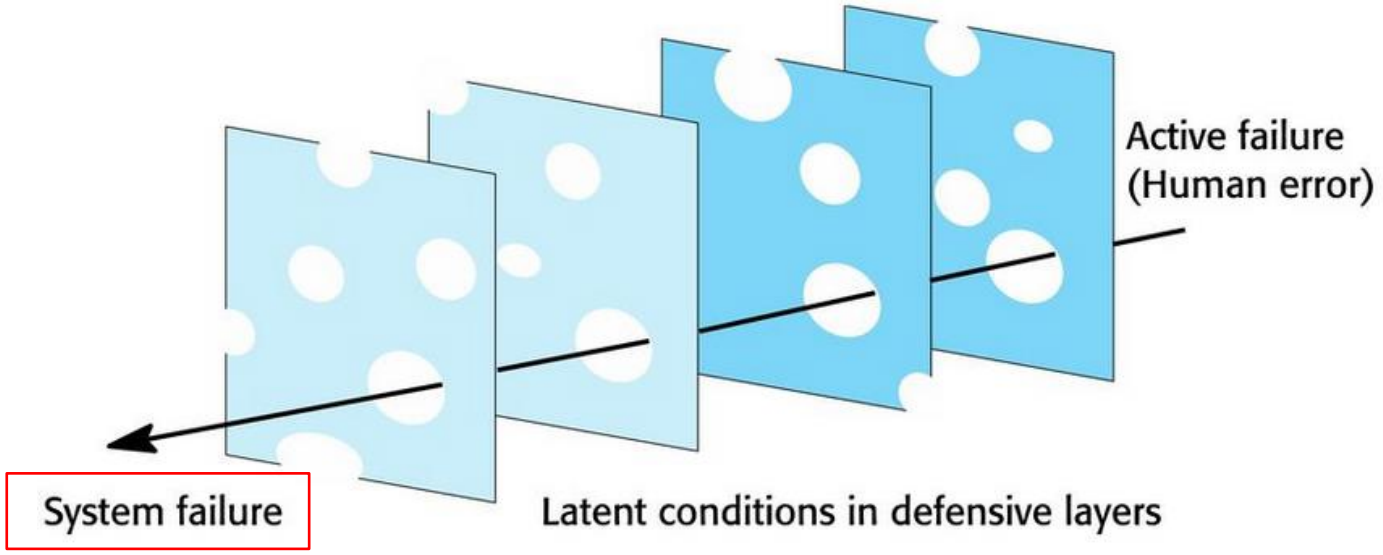


The reality: *Swiss cheese model*



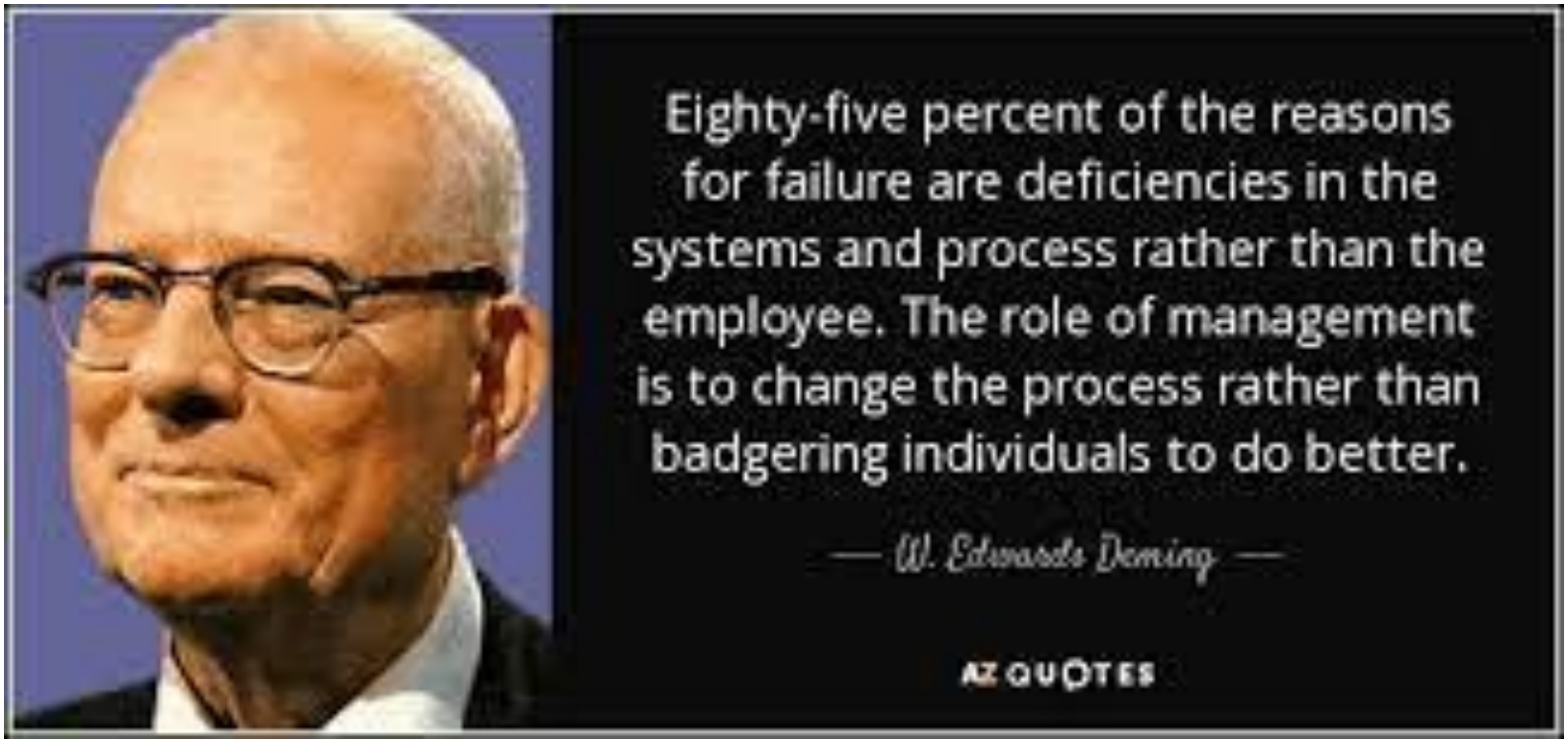
System resilience

- **Even defensive layers might have vulnerabilities**
  - They are like slices of Swiss cheese with holes in the layer corresponding to these vulnerabilities.
- **Vulnerabilities are dynamic**
  - The “holes”’ are not always in the same place and the size of the holes may vary depending on the operating conditions
- System failures occur and attacks are successful when the holes align, and all preventive defenses fail



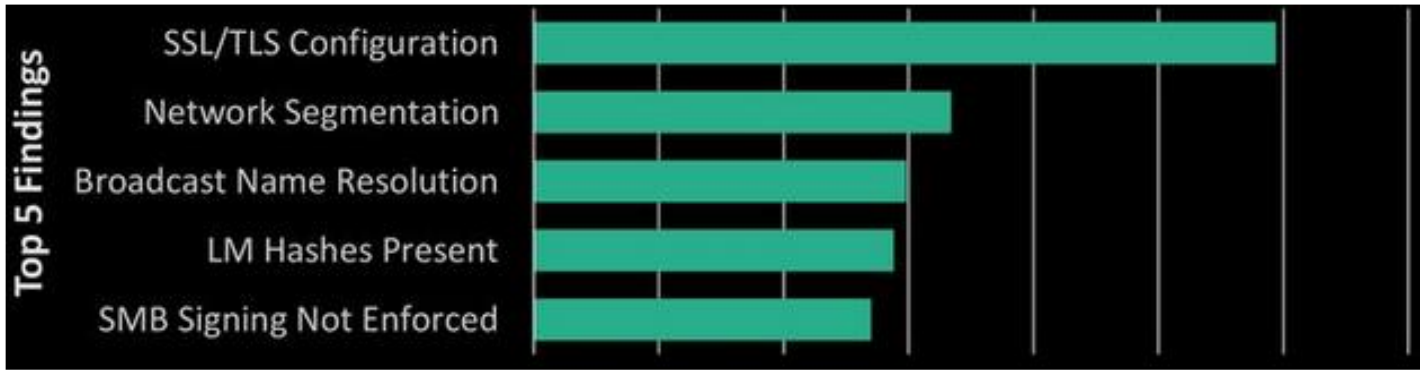
The important role of the human factors

People inevitably make mistakes (*human errors*) that sometimes lead to serious system and service failures



Top human errors in IT

Top 5 Vulnerability Themes	
Configuration Management	**40%**
Account Management	27%
Patch Management	13%
Authentication Weaknesses	7%
Software Lifecycle	5%



Human factors

In an organization, there are two ways to consider human error :

- **The person approach**: *errors are considered to be the responsibility of the individual*. “Unsafe acts”, such as an operator failing to engage a safety barrier, are a consequence of individual carelessness or reckless behavior
- **The systemic approach**: *the basic assumption is that people are fallible and will make mistakes*. People make mistakes because they are under pressure from high workloads, poor training or because of inappropriate system design.
  We know it and we should prevent them or limit the consequences though a holistic systemic approach

Systemic approach

- **Systems engineers** should assume that human errors will occur during system design, implementation and operation

- To improve the resilience of a system, **designers must think about the defences and barriers against human error that should be part of a system**

- **Can these barriers should be integrated into the technical components of the system (hardware/software barriers)?**
  - If not, could they be part of the processes, procedures, training and human guidelines for using the system (socio-technical barriers such as *playbooks*)?

Multiple defensive layers

You should use **redundancy** and **diversity** to create a set of **multiple defensive layers**, where each layer uses a different approach to deter attackers or to trap technical/human failures

Example: The multi-level approach of the ***Air Traffic Control system***

1) **Procedure**: Formalized recording procedures (*playbooks*)
2) **Technology**: Conflict alert system and system redundancy
3) **Human**: Collaborative checking and training

Continuous and adaptive approach

1. Identify main business assets and service requirements
2. Translate to IT assets
3. Identify the most critical services and components
4. Identify business and system threats (failures and attacks)

5. **Establish micro-perimeters**
6. "Defend" micro-perimeters
   - Redundancy (in-line equipment, transmission paths, power sources)
   - Technical defenses
7. Assess defenses
8. "Wash and rinse"
9. **REPEAT**

Why repetition?

PDCA (Deming's cycle)

Tips for performance and resiliency

- Check for bottleneck
- Check for single points of failure
- Check for interdependencies
- Keep it simple and stupid (KISS): so you can control

- **If it is not tested, do no rely on it**
- *Question*: how do you test availability with hundreds of loosely coupled services running on thousands of machines?

In summary

Technique	Performance	Availability
Replication	✔	✔
Partitioning (sharding)	✔	✔
Load-balancing	✔	
Watchdog timers		✔
Integrity checks		✔
Canaries		✔
Eventual consistency	✔	✔